



POLITECNICO
MILANO 1863

AIutami

AI-Moderated Voice Conference Platform

AUI Project Documentation

Group 8

Salvatore Pio Conte	salvatorepio.conte@mail.polimi.it
Simone Cosenza	simone.cosenza@mail.polimi.it
Sara Imparato	sara.imparato@mail.polimi.it

Abstract

AIutami is a web-based platform that introduces an AI moderator into vocal group discussions, designed to improve fairness, structure, and conversational flow. The system manages turn-taking to ensure only one participant speaks at a time, monitors conversational dynamics in real time, and intervenes contextually when discussions become unbalanced, stuck, or unfocused. Built on WebRTC for real-time audio streaming, automatic speech recognition for live transcription, and large language models for intelligent moderation, AIutami acts as a neutral facilitator: it listens, understands the flow, and guides the group toward productive outcomes without taking over the conversation. The platform addresses common challenges in group discussions—unequal participation, interruptions, and loss of focus—that current conferencing tools fail to handle in real time. By combining voice interaction with adaptive AI moderation, AIutami brings structure and fairness to collaborative scenarios such as study groups, team meetings, and decision-making activities.

Academic Year 2025/2026

Contents

1	The Team	3
2	Members' Contribution	4
3	Executive Summary	5
4	Requirements	6
4.1	Target Groups and Stakeholders	6
4.2	Stakeholder Needs	6
4.3	Context	7
4.4	Goals	7
4.5	Constraints	8
4.6	UNG Diagram	9
5	State of the Art	10
5.1	Related Works	10
5.2	Existing Products	11
5.3	Gap Analysis	11
6	Solution - UX Design	12
6.1	General Approach	12
6.2	User Workflows	12
6.2.1	Pre-Session Flow	13
6.2.2	In-Session Flow	14
6.3	User Scenarios	15
6.3.1	Scenario 1: A New User Registers	15
6.3.2	Scenario 2: A Registered User Logs In	15
6.3.3	Scenario 3: Creating a New Session	16
6.3.4	Scenario 4: Joining a Session	16
6.3.5	Scenario 5: Starting a Session	17
6.3.6	Scenario 6: Speaking During a Session	17
6.3.7	Scenario 7: Reserving the Next Turn	17
6.3.8	Scenario 8: Inactive User	18
6.3.9	Scenario 9: Ready to Conclude (Murder Mystery)	18
6.3.10	Scenario 10: Voting (Murder Mystery)	19
6.3.11	Scenario 11: Time Running Out	19
6.3.12	Scenario 12: Session Closure and Report	20
6.4	Conversational Interaction	20

7	Solution - Implementation	22
7.1	Deployment Architecture	22
7.2	Software Architecture	22
7.2.1	Functional Architecture	22
7.2.2	Implementation Architecture	23
7.3	AI Moderation System	23
7.3.1	Moderation Pipeline Overview	23
7.3.2	Trigger System	24
7.3.3	LLM Integration and Prompt Design	25
7.3.4	Intervention Control	26
7.4	Data Flow	26
7.4.1	Session Lifecycle	26
7.4.2	Speaking Turn Flow	27
7.4.3	Audio Flow	27
8	Value Proposition	28
8.1	Value for Users	28
8.2	Why It's a Good Solution	28
8.3	Competitive Landscape	29
9	Discussion and Future Work	30
9.1	Critical Reflection	30
9.2	Limitations	30
9.3	Future Developments	31
	Bibliography	32

1. The Team



Salvatore Pio Conte



Simone Cosenza



Sara Imparato

2. Members' Contribution

The following table summarizes each team member's role and responsibilities in the project.

Member	Role	Tasks
Salvatore Pio Conte	R	Backend architecture and implementation (Django, WebSocket, WebRTC, ASR integration), DevOps and deployment (Docker, Azure), contributor to UX design and prompt engineering
Simone Cosenza	R	Frontend development (React), UX design (user workflows, scenarios, interaction design), UNG model definition, testing and QA (functional testing, usability), prompt engineering for AI moderation
Sara Imparato	R	Frontend development (React), UX design (user workflows, scenarios, interaction design), UNG model definition, testing and QA (functional testing, usability), prompt engineering for AI moderation

Legend: R = Responsible (primary owner), C = Contributor (supporting role)

3. Executive Summary

Group conversations often fail to achieve their potential. Participants interrupt each other, some voices dominate while others remain unheard, and discussions frequently drift off track [1]. These dynamics undermine collaboration, making it difficult for groups to share knowledge effectively or reach clear decisions.

AIutami addresses these challenges through an AI-moderated discussion platform. The system allows users to create and join audio-based sessions through a web interface. Each session follows a structured workflow: participants gather in a lobby phase before entering the active discussion. During the conversation, speaking turns are regulated through a request-based mechanism that prevents overlap and ensures balanced participation. Visual feedback keeps participants informed about the current speaker and turn status, improving coordination and reducing confusion.

What distinguishes AIutami from traditional conferencing tools is its intelligent moderation layer. The platform combines real-time audio streaming, automatic speech recognition, and large language models to monitor conversational dynamics. When the discussion becomes unbalanced, stuck, or loses focus, the AI moderator intervenes contextually—prompting quieter participants, redirecting off-topic exchanges, or helping the group move forward.

Consider a Murder Mystery activity where participants must share clues and collaborate to identify the culprit. Without structure, a few voices dominate, clues get lost, and the group struggles to reason together. With AIutami, every participant gets heard, key points remain visible, and the group can build toward a shared conclusion. The same benefits apply to study groups debating complex topics, teams making strategic decisions, or workshops requiring structured input from all participants.

AIutami delivers concrete value: discussions that are fairer, more focused, and more likely to produce clear outcomes.

4. Requirements

This chapter describes the project requirements following the UNG model, which structures the analysis around Users (stakeholders), their Needs, and the system Goals that address those needs, all within a specific Context and subject to Constraints. The accompanying UNG diagram visualizes the relationships between these elements.

4.1 Target Groups and Stakeholders

The system addresses three levels of stakeholders based on their proximity to the platform and their role in the interaction.

Primary users are the direct participants in moderated discussions. This group includes students and researchers engaging in academic debates or group projects, colleagues collaborating in professional meetings, and participants in educational or entertainment activities.

Secondary users are those who oversee discussions without being the main speakers. This category includes teachers and trainers who monitor student participation, team leaders and managers who guide collaborative sessions, and facilitators who use the system as a support tool for group dynamics.

Tertiary users are individuals or organizations that do not interact directly with the platform but are affected by its adoption. These include educational institutions and companies that deploy the system to improve internal collaboration.

MVP Focus: For the purpose of this project, the implementation targets the Murder Mystery experimental paradigm [2], [3], where participants must collaborate vocally to solve a case. This context was chosen because it naturally exhibits the challenges the system aims to address: multiple voices competing to speak, risk of dominant participants, and the need for structured turn-taking to reach a shared conclusion.

4.2 Stakeholder Needs

The needs addressed by AIutami emerge from the typical dynamics of group vocal conversations, where communication imbalances, interruptions, and lack of inclusion often occur.

Feeling heard and included — Participants need equitable speaking opportunities without some voices monopolizing the conversation.

Communicating clearly and fluidly — Groups need to reduce interruptions, overlaps, and confusion that disrupt the natural flow of discussion.

Maintaining focus and productivity — Discussions often drift off-topic; participants need support to stay on track and reach shared conclusions.

Participating with confidence — Some participants feel hesitant to speak, especially in groups with dominant personalities. They need an environment that encourages balanced contribution.

Observing and improving group communication — Secondary users such as facilitators and teachers need tools to monitor participation patterns and guide the conversation effectively.

Ensuring transparency — Users need to understand when the AI is listening and how their contributions are being processed.

4.3 Context

The context describes where and how the system is used, articulated across four dimensions.

Organizational context — AIutami supports structured collaborative phases such as group study sessions, team meetings, brainstorming activities, and educational workshops. The system acts as a vocal facilitator during these interactions.

Physical context — The platform is a web-based application accessible through standard browsers. Users need a microphone or headset for audio input, a stable internet connection (approximately 10 Mbps), and low network latency (under 200ms) to ensure real-time responsiveness.

Temporal context — The system operates primarily in real-time mode, monitoring and intervening vocally during active discussions. At the end of each session, a summary report is generated to capture key outcomes.

Social context — AIutami is designed for small groups of 2 to 8 participants. These groups may have equal roles (peers collaborating) or differentiated roles (teacher-students, team leader-colleagues). The shared objective is typically collaborative decision-making, learning, or structured discussion.

MVP Scope: The current implementation focuses on the Murder Mystery experimental scenario with 3 participants collaborating to solve a case through structured vocal interaction.

4.4 Goals

The goals represent the functional objectives that the system must achieve to address user needs. While needs describe what users require, goals describe what the system does to satisfy those requirements.

Manage turn-taking fluidly — The system regulates speaking turns through a request-based mechanism, ensuring only one participant speaks at a time and preventing overlaps. Visual feedback indicates the current speaker and turn status.

Identify participants through the turn system — Rather than using voice recognition to identify speakers, the system tracks who is speaking based on who holds the current turn. This approach associates each transcribed contribution with the correct participant.

Provide real-time vocal feedback — The AI moderator intervenes with synthesized voice messages when discussions become unbalanced, stuck, or unfocused. These interventions prompt quieter participants, redirect off-topic exchanges, or help the group move forward.

Summarize key points — The system generates vocal summaries during and at the end of sessions, highlighting main discussion points and outcomes. A final report preserves these summaries for future reference.

Monitor participation levels — The system tracks speaking time and turn distribution across participants, enabling balanced contribution and providing data for secondary users to assess group dynamics.

4.5 Constraints

Several factors constrain the system’s design and performance.

Potential latency in the processing chain — The system relies on a sequential pipeline: speech recognition (ASR), language model processing (LLM), and voice synthesis (TTS). Each step introduces potential delays, particularly if the LLM response is slow. While current performance is acceptable, network conditions or service load could affect real-time responsiveness.

Speech recognition accuracy — The ASR service may produce transcription errors, particularly with background noise, accents, or unclear speech. While occasional errors are acceptable, they can affect the quality of summaries and the AI’s understanding of the discussion.

LLM behavioral limitations — The language model powering the AI moderator has limited customization. Content moderation policies and inherent model behaviors may occasionally produce responses that are not perfectly suited to the moderation context.

Cloud-based audio processing — Audio data is transmitted to external cloud services (Azure Speech) for transcription. While these services follow industry security standards, users should be aware that their voice data is processed externally. The system does not store raw audio or full transcriptions—only session summaries are retained for the final report.

4.6 UNG Diagram

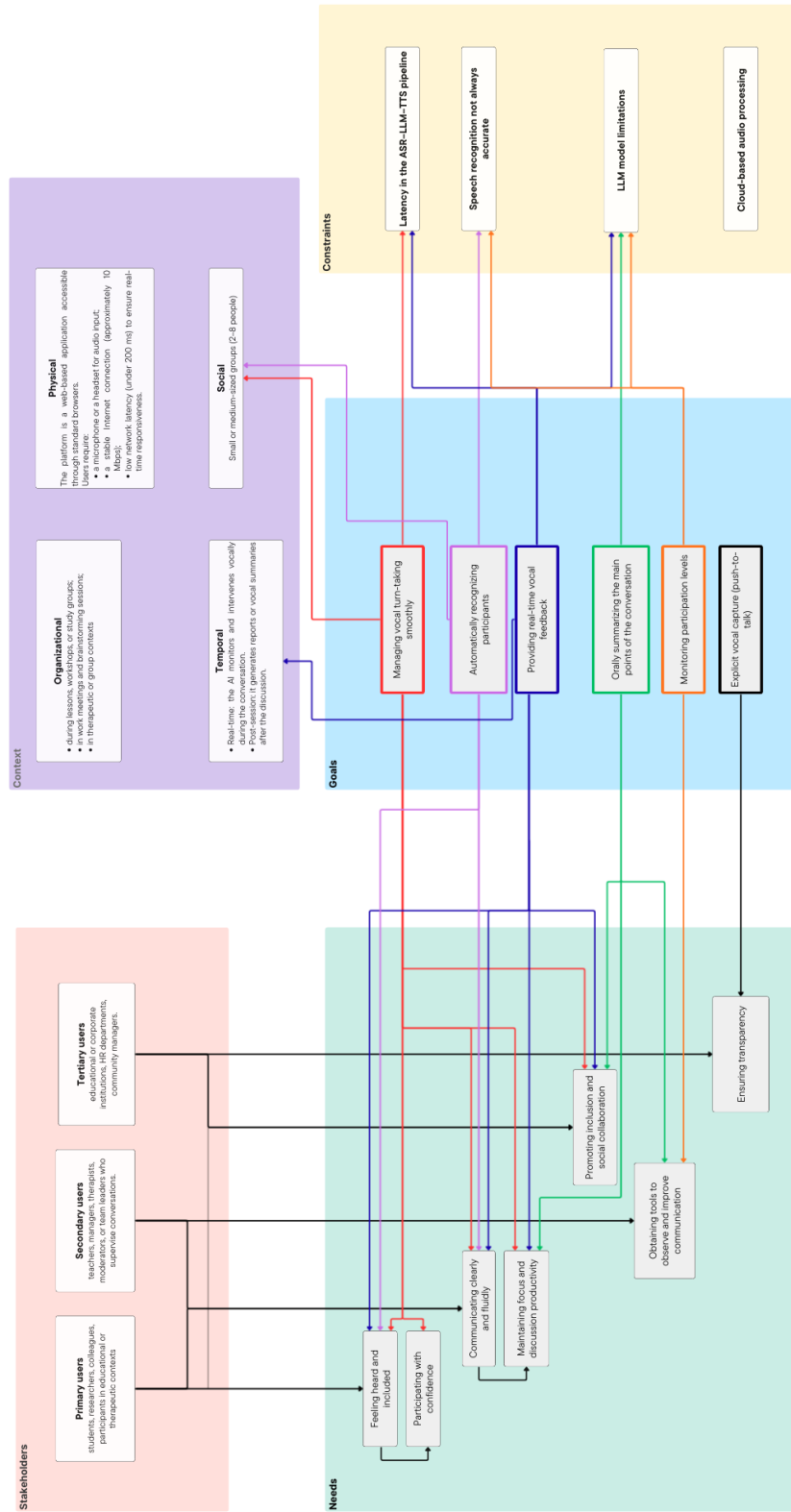


Figure 1: UNG Diagram showing the relationships between stakeholders, needs, goals, context, and constraints.

5. State of the Art

This chapter analyzes related academic works, existing commercial products, and identifies open research directions relevant to the AIutami project.

5.1 Related Works

LLMs in Multi-User Contexts — Research on LLM-based conversational systems has historically focused on dyadic interactions. The Multi-User Chat Assistant (MUCA) framework from Microsoft Research represents one of the first systematic attempts to address multi-user contexts [4]. MUCA introduces the 3W model that formalizes three fundamental dimensions for group chatbots: What (response content), When (timing or strategic silence), and Who (addressing an individual, subgroup, or entire group). The framework identifies recurring challenges in group conversations: stuck discussions, multi-thread management, reactivity control, participation equity, and conflict resolution. Its modular architecture combines topic generation, dialog analysis, and strategy arbitration. Results show improvements in intervention timing and perceived quality compared to single-prompt approaches, though the framework operates exclusively in the text domain.

AI-Based Moderation and Facilitation — Group discussion moderation has traditionally relied on human facilitators or rigid rule-based systems. The thesis “Multiuser Conversational Agents Based on Large Language Models” [5] represents pioneering work exploring LLM-based moderation. The study analyzes a GPT-3.5 moderator using the Hidden Profile experimental paradigm—a collaborative decision-making task where participants possess partial, unique information, requiring effective discussion to reach the correct solution. Results show that the AI moderator influences conversation structure and participation patterns, though it does not significantly improve decision accuracy. The work identifies key limitations: task-agnostic moderators may suggest unrealistic actions, participant motivation affects outcomes, and standard evaluation metrics are lacking. The thesis concludes that LLM moderation is promising but requires greater task contextualization. A subsequent study, “Large Language Models as Facilitators in Multi-user Conversations” [6], extends these findings using a GPT-4o facilitator with minimal prompt engineering. Participants with the facilitator achieve higher individual accuracy. The analysis identifies emergent functions: conversational regulation, task-specific guidance, bias reduction, and time management. However, occasional hallucinations and premature time pressure remain problematic.

LLM Limitations in Multi-Turn Conversations — The paper “LLMs Get Lost in Multi-Turn Conversation” [7] addresses a critical limitation relevant to moderation systems. The study demonstrates that LLM performance degrades as conversational turns increase, even without semantic ambiguity. This loss of conversational state tracking manifests through forgotten constraints, contradictions with past statements, and locally

correct but globally incoherent responses. Neither larger context windows nor bigger models resolve the issue. The authors attribute this to structural factors: LLMs lack explicit dialogue state representation and treat context as unstructured text. The implication is clear: robust conversational systems require structured memory and explicit state tracking beyond pure prompting approaches.

5.2 Existing Products

Commercial video conferencing platforms such as Zoom, Microsoft Teams, and Google Meet provide real-time audio communication but lack intelligent moderation. Turn-taking is manual, participation balance is not monitored, and no AI-driven facilitation exists.

Transcription services like Otter.ai focus on documentation rather than active intervention. Conversation intelligence platforms such as Gong operate retrospectively on recorded calls rather than in real-time.

No commercial product currently combines real-time voice conferencing with AI-based moderation for group discussions.

5.3 Gap Analysis

The literature review reveals open directions in LLM-based conversational systems.

Modality — Existing research focuses on text-based conversations, while commercial products handle voice without intelligent moderation. The intersection—real-time voice moderation—remains largely unexplored.

Context Awareness — Research indicates that task-agnostic moderators produce suboptimal interventions. Approaches balancing LLM flexibility with domain awareness represent an open area.

Architectural Robustness — Studies show that prompt-only systems struggle with state coherence in extended conversations. Integrating structured memory with conversational AI is an active research direction.

AIutami positions itself within these open spaces, exploring an architecture that combines voice-based interaction, contextual moderation, and explicit state management.

6. Solution - UX Design

6.1 General Approach

The UX design of AIutami follows a core principle: minimize interface complexity so that participants can focus entirely on the conversation. As a real-time voice platform, every interface element is designed to be immediately understandable without prior instruction.

The interaction follows a sequential phase model—registration, session creation or joining, lobby, active discussion, conclusion, and session closure—each with a dedicated interface that displays only the actions available at that stage. This approach reduces cognitive load: users never face unnecessary choices or ambiguous options.

During the active discussion phase, the interface centers on a single primary element: the push-to-talk button. The choice of push-to-talk over an always-open microphone is deliberate: it makes the act of “taking the floor” an explicit gesture, aligning the digital interaction with the natural dynamics of a moderated conversation. Visual indicators display who is currently speaking, who has reserved the next turn, and the AI moderator’s status.

The AI moderator is integrated into the experience as an additional participant rather than a separate control panel. It intervenes vocally when needed—prompting quieter users, redirecting off-topic exchanges, or signaling phase transitions—without requiring users to interact with it through commands or text input. Moderation happens transparently, driven by real-time analysis of the conversation’s transcription and participation patterns.

6.2 User Workflows

This section describes the two main workflows that define the user experience in AIutami. The first covers the steps from account creation to the start of a moderated discussion; the second describes the interaction dynamics during and after the discussion.

6.2.1 Pre-Session Flow

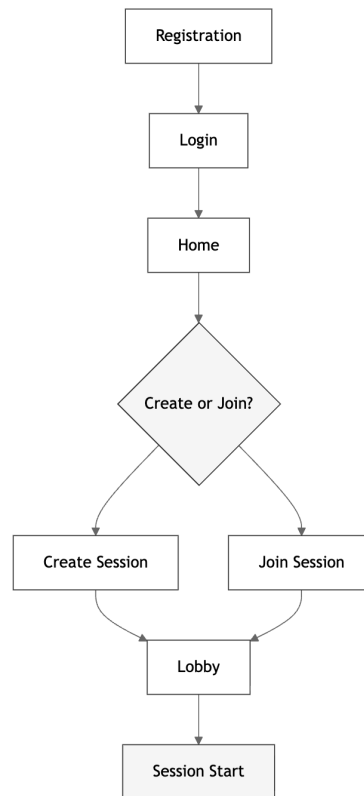


Figure 2: Pre-Session Workflow

The pre-session workflow covers all steps from account creation to the start of a moderated discussion.

1. **Registration** — The user creates an account by providing name, surname, email, username, and password. The account is immediately active upon successful registration.
2. **Login** — The user authenticates with email and password, obtaining access to the platform.
3. **Home** — The user accesses the main view, which presents three options: create a new session, join an existing session via invitation link, or browse past sessions.
4. **Create Session** (path A) — The user selects a discussion context (Murder Mystery, Academic, Therapeutic, or Workplace), enters a session title, and confirms. The system creates the session and redirects the user to the lobby as host.
5. **Join Session** (path B) — The user pastes an invitation link received from a host. The system validates the link and adds the user to the session's lobby.
6. **Lobby** — All participants wait in the lobby. The host can see the participant count and, once the required number of participants has joined, starts the session.
7. **Session Start** — The AI moderator delivers an introductory message explaining the rules and the discussion begins.

6.2.2 In-Session Flow

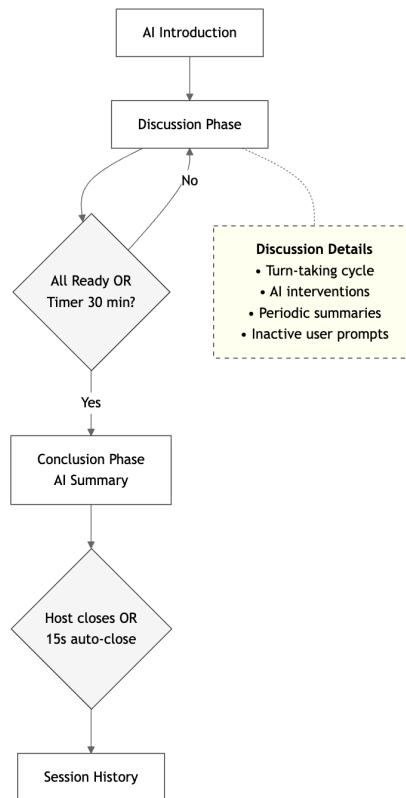


Figure 3: In-Session Workflow

The in-session workflow describes the interaction dynamics from the start of the discussion to session closure.

1. **AI Introduction** — The moderator welcomes participants, explains the interaction rules, and opens the discussion.
2. **Discussion Phase** — Participants engage in conversation through a turn-taking system: one speaks via push-to-talk while others can reserve the next turn with an 8-second priority window. Throughout the discussion, the AI moderator monitors participation and intervenes when needed—delivering periodic summaries, prompting inactive users, or addressing collective silence.
3. **Conclusion Phase** — Triggered when all participants declare readiness or when the 30-minute timer expires. The moderator delivers a vocal summary of the discussion. In the Murder Mystery context, this is followed by a voting phase where participants select the suspect they believe is guilty.
4. **Session Closure** — The host closes the session manually, or the system closes it automatically after 15 seconds. A final report is generated.
5. **Session History** — The session appears in each participant's history, where the report can be viewed and downloaded as PDF.

6.3 User Scenarios

This section presents a series of user scenarios that illustrate the main interactions with the platform, from registration to session closure.

6.3.1 Scenario 1: A New User Registers

Luca, a new user interested in participating in moderated discussions on AIutami, decides to sign up. He accesses the homepage and selects the “Sign up” option.

He fills in the registration form with his first name, last name, email address, username, and password. After submitting the form, AIutami verifies that the chosen username and email address are not already associated with an existing account. Upon successful validation, Luca’s account is immediately created and active.

Luca is redirected to the login page, where he enters his credentials to access the platform for the first time. Once authenticated, he reaches the home page, from which he can create a new voice session, join a session via an invitation link, or browse the history of completed sessions.

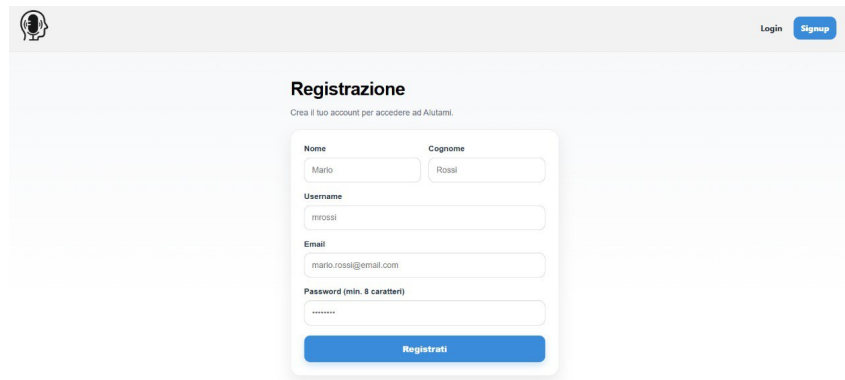
The image shows a web browser window with a registration form titled "Registrazione". At the top right of the browser window, there are "Login" and "Signup" buttons. The form itself has a subtitle "Crea il tuo account per accedere ad AIutami." and contains several input fields: "Nome" (with the value "Mario"), "Cognome" (with the value "Rossi"), "Username" (with the value "mrossi"), "Email" (with the value "mario.rossi@email.com"), and "Password (min. 8 caratteri)". A blue "Registrati" button is at the bottom of the form.

Figure 4: Sign up interface for new user registration.

6.3.2 Scenario 2: A Registered User Logs In

Marco, an existing user of AIutami, wants to access his account. From the homepage he selects “Log in” and enters his email and password. The system verifies his credentials and, once authenticated, redirects him to the home page. From here he can view active sessions, create a new one, join an existing session through an invitation link shared by another participant, or browse the history of past sessions.

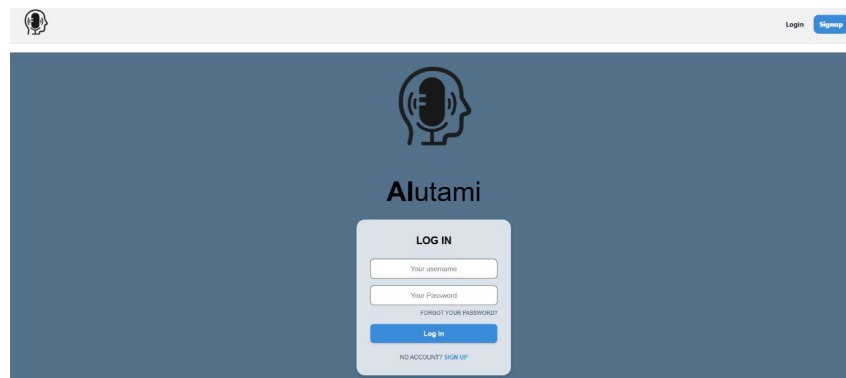


Figure 5: Login interface for existing users.

6.3.3 Scenario 3: Creating a New Session

Luca, who has just logged in to AIutami, wants to start a new session to discuss with other users. From the home page, he selects “Create new session” and is redirected to the session configuration page.

Here he chooses one of the available discussion contexts—Murder Mystery, Academic, Therapeutic, or Workplace—and enters a title for the session. Once confirmed, the system creates the session and redirects Luca to the lobby as host.

From the lobby, Luca generates an invitation link and shares it with the other participants he wants to invite. The lobby displays in real time the list of participants who have joined. Luca waits until the required number of participants has connected before starting the discussion.

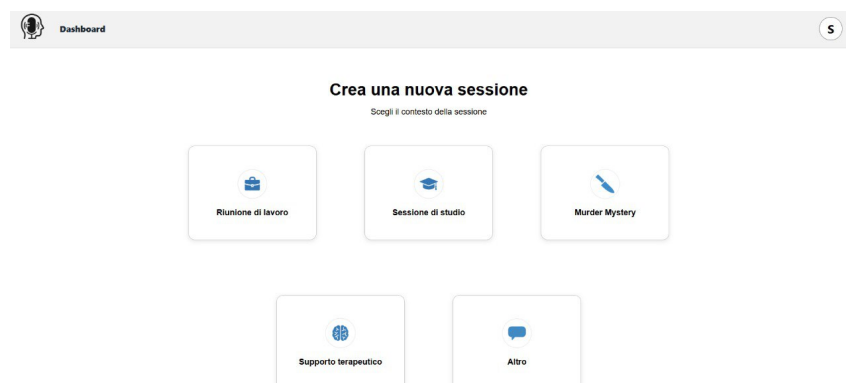


Figure 6: Session creation interface with context selection.

6.3.4 Scenario 4: Joining a Session

Giovanni, who is already logged in to AIutami, wants to join a session created by his friend Luca. From the home page, he selects “Join with a link” and is redirected to the join page, where he finds a text field to paste the invitation link received from Luca. After pasting it, he confirms by clicking “Join session.”

The system validates the link and checks that the session is still in the lobby phase and has not reached its maximum number of participants. If successful, Giovanni is redirected to the lobby, where he can see the list of participants already connected and wait for the host to start the discussion.



Figure 7: Join session interface where users paste the invitation link.

6.3.5 Scenario 5: Starting a Session

Marco, the host of a Murder Mystery session on AIutami, is in the lobby together with the invited participants, Luca and Giovanni. In this context, the number of participants is fixed at three, so the interface displays a counter indicating how many users have joined.

When the counter reaches the required number, Marco selects the “Start session” button, visible only to him as host. The session enters its active phase and all participants are redirected to the discussion interface.

The AI moderator takes the floor to introduce the session: it welcomes the participants, explains the interaction rules, and opens the discussion.

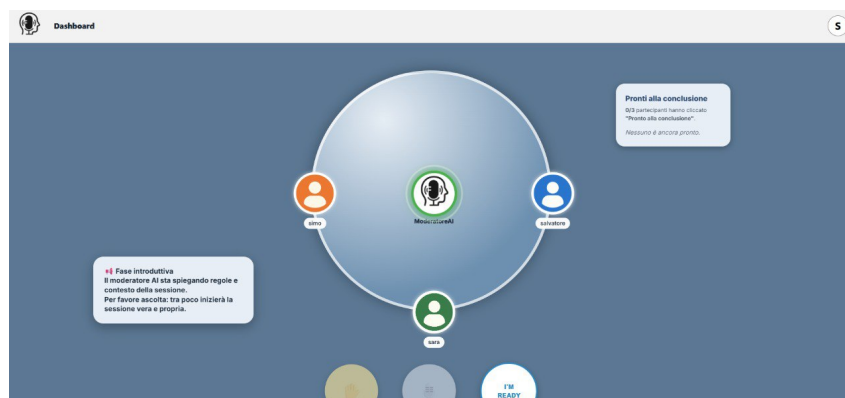


Figure 8: Session interface showing participants and AI introducing.

6.3.6 Scenario 6: Speaking During a Session

Marco, a participant in an active session on AIutami, wants to open the conversation. He presses the push-to-talk button to activate his microphone. The interface visually indicates that his turn is active, while the other participants can only reserve their turn to speak next.

Marco shares his thoughts and, once finished, selects the “End turn” button, releasing the floor. From that moment, any participant can press the push-to-talk button to take the next turn.

6.3.7 Scenario 7: Reserving the Next Turn

Giovanni, a participant in the same session, wants to speak while Marco is talking. From his interface, he presses the “Reserve turn” button, which is available only when another participant holds the floor.

The system registers his request. When Marco ends his turn, Giovanni receives an 8-second priority window during which only his push-to-talk button is active. Giovanni presses it within this interval and begins speaking. After he finishes and ends his turn, the floor becomes available to all participants again.

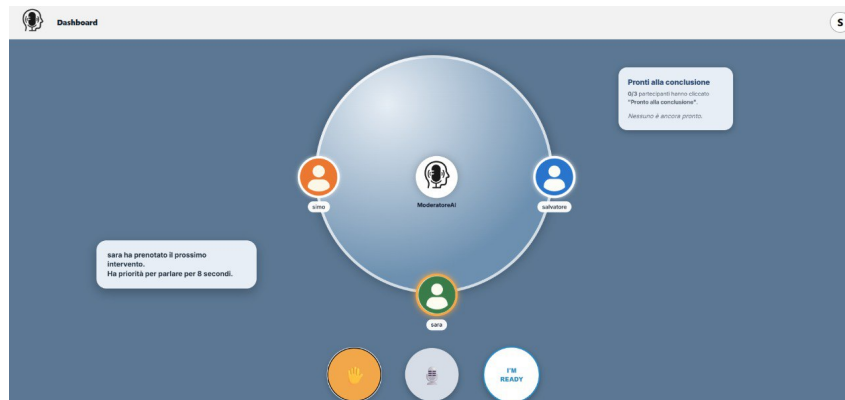


Figure 9: Discussion interface showing a turn reservation request.

6.3.8 Scenario 8: Inactive User

Giovanni is in a session together with Luca and Marco. The discussion is about a project update and proceeds at a steady pace: Luca and Marco alternate speaking turns while Giovanni listens in silence without ever pressing the push-to-talk button.

After several minutes of inactivity, a private text notification appears on Giovanni's screen, inviting him to participate in the conversation. Giovanni reads the message but chooses not to speak and lets the others continue.

After further minutes of inactivity, the AI moderator intervenes vocally, addressing Giovanni by name and encouraging him to share his thoughts with the group. Giovanni decides to accept the invitation, presses the push-to-talk button, and contributes his perspective to the discussion.

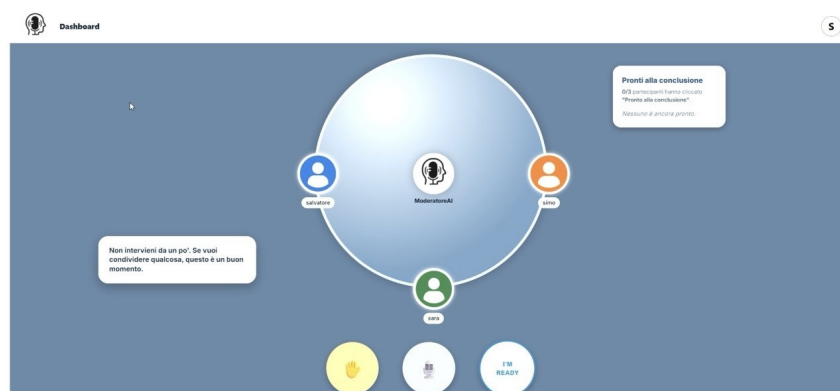


Figure 10: Private notification displayed to an inactive user, prompting participation.

6.3.9 Scenario 9: Ready to Conclude (Murder Mystery)

Chiara, a participant in a Murder Mystery session, has been collaborating with Matteo and Silvia to identify the culprit. After several exchanges, Chiara believes she has found a possible solution and presses the “Ready to conclude” button.

The system registers her action and the AI moderator announces to the group that Chiara is ready to propose a conclusion. Chiara continues participating in the discussion while Matteo and Silvia keep deliberating.

Eventually, Matteo also presses the button, and the moderator announces his declaration. When Silvia, the last remaining participant, presses “Ready to conclude” as well, the moderator announces that all participants are ready and the session transitions to the conclusion phase. The AI moderator delivers a vocal summary of the discussion, recapping the main points and clues that emerged.

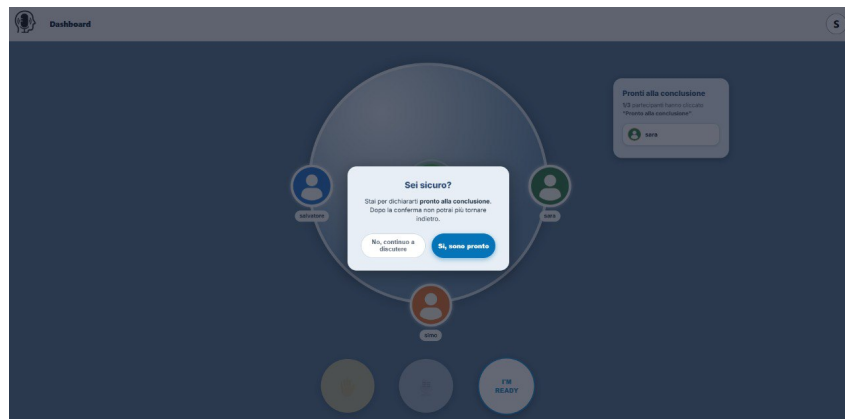


Figure 11: Discussion interface with the “Ready to conclude” button.

6.3.10 Scenario 10: Voting (Murder Mystery)

After the AI moderator has delivered the discussion summary, the session enters the voting phase. Each participant sees a voting interface presenting the list of suspects. Chiara, Matteo, and Silvia must each select the character they believe is the culprit.

Chiara casts her vote first. The system registers her choice and notifies the group that she has voted, without revealing her selection. Matteo and Silvia follow. Once all three participants have voted, the system reveals the results: each participant’s choice, the correct answer, and whether the group successfully identified the culprit.

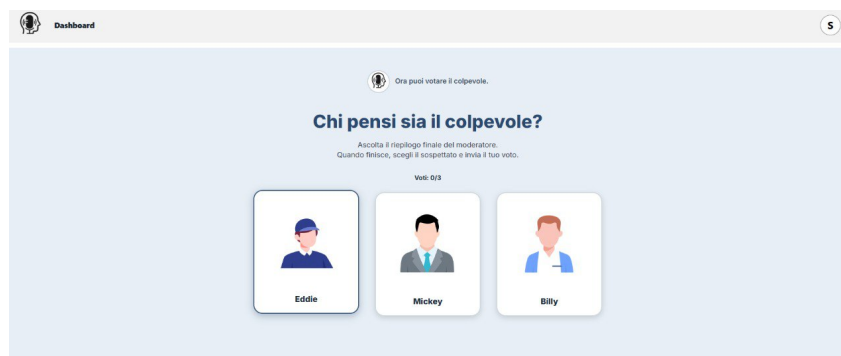


Figure 12: Voting interface showing the list of suspects for selection.

6.3.11 Scenario 11: Time Running Out

During an active session on AIutami, Giulia is participating in a Workplace discussion. The maximum session duration is 30 minutes, visible in the interface.

When the elapsed time reaches 25 minutes, a text notification appears on each participant’s screen, informing them that approximately five minutes remain before the conclusion phase begins. Giulia reads the message and dismisses it, returning to the discussion.

When the 30-minute mark is reached, the AI moderator intervenes vocally to announce that the discussion time has ended. The session transitions to the conclusion phase: the moderator delivers a vocal summary of the key points and outcomes of the conversation, and the session proceeds to its closing steps.

6.3.12 Scenario 12: Session Closure and Report

After all participants have cast their votes in the Murder Mystery session, Marco, as host, selects the “Close session” button. The system generates a final report containing the session details, voting results, and an analysis of the discussion. Marco is redirected to the session history.

Later, Sara, one of the participants, wants to share the session outcome with her professor. From the home page, she selects “Session history” and locates the completed session. On the detail page she can view the session creator, participants, duration, voting results, and the summary generated by the AI moderator. She downloads the report as PDF and sends it to her professor.

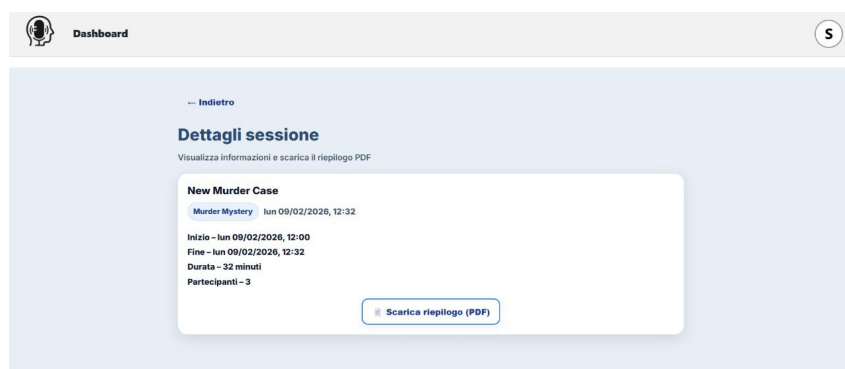


Figure 13: Session details and button to generate the report.

6.4 Conversational Interaction

The AI moderator in AIutami acts as a neutral vocal facilitator that participates in the discussion alongside human users. It does not express opinions on the topic being discussed, nor does it take sides. Its role is exclusively procedural: ensuring balanced participation, maintaining focus, and guiding the group through the session’s phases.

The moderator communicates through synthesized voice messages delivered via the same audio channel used by human participants. This design choice makes the moderator’s presence natural within the conversation flow, rather than relying on text overlays or separate notification systems. When the moderator speaks, the interface reflects this through the same visual indicators used for human speakers. All interventions follow a principle of minimal intrusiveness: the moderator speaks only when necessary, uses brief and indirect language, and never interrupts a participant who is speaking.

From the user’s perspective, the moderator’s interventions fall into distinct categories. **Session-level announcements** occur at key phase transitions: the moderator introduces the session at the start, warns participants when discussion time is running low,

and delivers a closing summary when the session moves to its conclusion phase. **Periodic summaries** provide a brief vocal recap of the main points at regular intervals, optionally including a gentle redirection if the conversation has drifted or participation has become unbalanced. **Individual engagement** addresses prolonged silence from a specific participant: first through a private text notification visible only to that user, then, if the silence continues, through a vocal prompt addressing them by name. **Collective silence** handling activates when no participant takes the floor for a prolonged period, encouraging someone to restart the discussion. Finally, **contextual interventions** are triggered after speaking turns when significant issues are detected—such as monopolization, aggressive tones, or a direct request for help. Minor deviations and civil disagreements do not trigger an intervention.

7. Solution - Implementation

7.1 Deployment Architecture

AIutami is deployed on a single Azure Virtual Machine hosting all services through Docker Compose. The deployment architecture comprises three containers. The **web** container runs the Django application server served by Daphne (ASGI server), exposing port 8000/TCP for HTTP and WebSocket traffic and ports 10000–10050/UDP for WebRTC media traffic (RTP/RTCP via aiortc). The **db** container runs PostgreSQL 16 on port 5432, with persistent data stored on a Docker volume. The **redis** container runs Redis 7 on port 6379, used as cache and message broker for Django Channels. The web container depends on both db and redis, ensuring sequential startup.

In addition to local services, the system communicates with three external Azure services over HTTPS: Azure Speech-to-Text for speech transcription, Azure OpenAI for moderation decisions, and Azure Speech TTS for moderator voice synthesis.

7.2 Software Architecture

7.2.1 Functional Architecture

The system is decomposed into six functional modules, each responsible for a distinct concern. **Session Management** controls the session lifecycle through four states (Lobby, Active, Conclusion, Closed), manages participants, invitations, and voting. **Turn Management** coordinates speaking turns through a state machine with four states (Idle, Human Speaking, AI Speaking, AI Introducing), enforces that only one participant speaks at a time—following the principles of conversational turn-taking [8]—and manages a priority reservation window for the next speaker. **Audio Transport** handles WebRTC peer connections and server-side audio forwarding; the server receives audio from the current speaker and forwards it to all other participants without mixing, excluding the speaker from their own stream to prevent echo. **Speech Recognition** performs real-time speech-to-text transcription of the current speaker’s audio using Azure Speech-to-Text; transcription is gated by the turn state and only runs when the user is the active speaker. **AI Moderation** evaluates whether the AI moderator should intervene after each human turn, using a trigger system and LLM calls to decide the content and timing of interventions. **Text-to-Speech** converts AI moderator text responses into audio using Azure Speech TTS, which is then injected into the audio stream via the Audio Transport module.

The modules interact in a pipeline: Turn Management acts as the central coordinator. When a speaker’s turn ends, it triggers Speech Recognition to finalize the transcript,

passes it to AI Moderation for evaluation, and if the AI decides to intervene, Text-to-Speech generates the audio delivered through Audio Transport.

7.2.2 Implementation Architecture

The implementation follows a **service-oriented design**, where business logic is encapsulated in dedicated service classes separate from the communication layer. `TurnManager` manages the turn state machine, `ModerationOrchestrator` coordinates moderation decisions, and `ASRStreamManager` controls speech recognition streams. WebSocket consumers act as coordinators: they receive client messages, delegate to services, and broadcast results. This separation keeps business rules testable independently from the transport layer.

A **dual storage strategy** assigns each data category to the backend best suited for it. PostgreSQL stores persistent data that must survive restarts: sessions, participants, invitations, votes, and transcripts. Redis stores high-frequency ephemeral state that requires low-latency access: turn state, moderation state, and ASR transcript cache. This split avoids the overhead of database transactions for state that changes multiple times per second during active sessions.

WebSocket consumers are **asynchronous**, built on Django Channels, since they must handle concurrent connections and I/O-bound operations like network broadcasting. However, the service layer remains **synchronous** for simplicity and easier reasoning about state. The bridge between the two layers is Django's `database_sync_to_async` wrapper, which allows synchronous service calls to run safely within the asynchronous consumer context.

Every state change is propagated to all connected clients through an **event-driven broadcasting** mechanism. When a participant joins, requests a turn, or when the AI moderator intervenes, the corresponding consumer broadcasts an event to the session's channel group. Clients subscribe to three WebSocket endpoints—sessions, turns, and WebRTC—each carrying its own category of events. This decouples the sender from the receivers and ensures all clients maintain a consistent view of the session state.

Finally, rather than establishing direct peer-to-peer connections between participants, all audio flows through the server following a **server-side forwarding** pattern. The server receives PCM audio frames from the current speaker and forwards them to every other participant's outbound track, explicitly excluding the speaker from their own stream to prevent echo. This server-centric approach enables two key features: ASR gating, where the server feeds only the current speaker's audio to the transcription service, and AI audio injection, where the moderator's synthesized voice is delivered through a virtual audio track to all peers.

7.3 AI Moderation System

7.3.1 Moderation Pipeline Overview

The AI moderation system evaluates whether the moderator should intervene after each human speaking turn. The pipeline executes during a moderation window in which no participant can take the floor, ensuring the AI's evaluation is based on a complete turn.

The pipeline follows a fixed sequence. First, the `ModerationOrchestrator` loads the current moderation state from Redis, which includes a running summary of the discussion,

the number of turns per participant, and timestamps of previous AI interventions. Second, it evaluates the trigger engine to determine if any rule-based condition requires a hard action (such as a forced summary) or if any static messages need to be delivered. Third, it calls the LLM via `ModerationService` to analyze the last speaker's transcript against the discussion summary. The LLM returns a structured JSON response containing an updated summary, a boolean indicating whether the AI should speak, a proposed message, a reason classification, and an intervention score. Fourth, the backend applies its own filtering rules on top of the LLM's proposal—checking cooldown timers, score thresholds, and session phase—before producing a final decision. This decision contains the list of static messages to deliver, whether the AI should speak, the AI message content, and whether the session should transition to the conclusion phase.

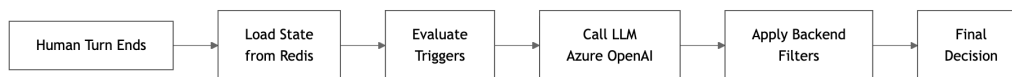


Figure 14: Moderation pipeline sequence. After each human turn, the system loads state from Redis, evaluates triggers, calls the LLM, applies backend filters, and produces a final moderation decision.

7.3.2 Trigger System

The moderation system uses a trigger engine that evaluates rule-based conditions independently from the LLM. Triggers are divided into three categories based on how they are activated.

Turn-based triggers are evaluated at the end of each human speaking turn. The primary trigger in this category is the periodic summary: after every six human turns, the system forces a summary intervention where the AI recaps the discussion so far and optionally corrects participation imbalances such as monopolization or exclusion of quiet participants.

Time-based triggers are evaluated by a background loop that runs every five seconds, independently from speaking turns. These include: a silence trigger that fires when

no activity is detected for a configurable period, prompting participants to resume the discussion; an inactive user trigger with two escalation levels—a private text notification after five minutes of individual silence, followed by a public voice solicitation after ten minutes, with a maximum of two voice solicitations per user to avoid being intrusive; and session duration milestones at twenty-five and thirty minutes. The twenty-five minute mark delivers a text-only notification to the frontend, which activates a visual countdown timer. The thirty-minute mark triggers a voice announcement and automatically transitions the session to the conclusion phase. Importantly, time-based triggers are suppressed while someone is speaking—they are re-evaluated at the next cycle when the session is idle.

Event-based triggers respond to participant actions. When a participant presses the “ready to conclude” button, the system generates a contextual message that varies based on how many participants are ready: a general notification for early votes, an urgency message when only one vote is missing, and a transition announcement when all participants have voted.

Each trigger produces either a static message (delivered as text or voice) or a hard action that overrides the normal LLM evaluation path.

7.3.3 LLM Integration and Prompt Design

The system integrates with Azure OpenAI through a structured request-response protocol. Each LLM call sends a system prompt defining the moderator’s role and a user message containing a JSON object with the current discussion state. The LLM responds with a JSON object following a strict contract, which the backend parses and validates.

The LLM operates in three modes, each with a dedicated system prompt. In **normal mode**, the LLM acts as a neutral facilitator. It receives the discussion summary, the last speaker’s transcript, and the turn count per participant. It evaluates whether an intervention is needed based on five criteria: monopolization of the conversation, exclusion of quiet participants, off-topic drift, interpersonal conflict, or a direct request for help from a participant. It returns an intervention score between 0 and 1, and only proposes to speak when the score reaches 0.7 or above. In **forced summary mode**, the LLM generates a periodic recap of the discussion. It combines two responsibilities: detecting and gently correcting participation imbalances, and summarizing the key points and clues that have emerged so far. In **forced conclusion mode**, the LLM produces a closing message that summarizes the entire discussion, provides voting instructions, and thanks the participants.

A central mechanism across all modes is the **running summary**. The LLM receives the current summary and the latest transcript, and returns an updated summary that incorporates the new information. This summary persists in Redis across turns, allowing the LLM to maintain context of the full discussion without receiving the entire transcript history in each call—keeping token usage bounded regardless of session length.

The system includes a fallback mechanism: if the Azure API call fails or returns unparseable output, a local fallback generates a safe default response—no intervention in normal mode, a basic recap in summary mode, and a template closing message in conclusion mode.

7.3.4 Intervention Control

Beyond the LLM’s own judgment, the backend enforces several control mechanisms that regulate when and how the AI moderator can speak.

The primary mechanism is a **cooldown timer**: after each normal intervention, the system blocks further normal interventions for sixty seconds. This prevents the AI from dominating the conversation with frequent remarks. The cooldown can be bypassed only when the LLM classifies the situation as a conflict or a direct user request—cases where delayed intervention could be harmful. Forced modes (summary and conclusion) are exempt from the cooldown entirely, as they serve structural purposes that must execute regardless of recent activity.

A **moderation lock** ensures mutual exclusion between the moderation pipeline and speaking turns. When a human turn ends and the pipeline begins evaluating, a flag is set that blocks any participant from requesting a new speaking turn. This guarantees that the AI’s evaluation is not disrupted by concurrent speech and that the subsequent voice intervention, if any, is delivered without interruption. The flag is cleared only after all static messages and the optional AI voice message have been fully delivered.

The system also manages an **intro message** at session start. When a session transitions to the active state, a pending flag is set in Redis. Before the first human turn is allowed, the AI moderator delivers a welcome message that greets participants by name, explains the interaction mechanics (how to speak, how to reserve, how to conclude), and sets the session duration.

Finally, a **pending message queue** handles situations where the AI needs to speak but the floor is occupied. If a time-based trigger fires while someone is speaking, the message is enqueued in a Redis FIFO queue. When the turn returns to idle, the queue is flushed and all pending messages are delivered in order. The queue includes deduplication to avoid repeating identical messages and blocks new entries once a conclusion-triggering message has been enqueued.

7.4 Data Flow

7.4.1 Session Lifecycle

A session progresses through four states in a fixed sequence: Lobby, Active, Conclusion, and Closed.

The transition from Lobby to Active occurs when the host explicitly starts the session via REST endpoint. The system verifies that the required number of participants has joined—three for Murder Mystery—then initializes the moderation timers and sets the turn state to AI Introducing, blocking all participant interactions until the moderator’s welcome message is delivered.

The Active phase ends either when the thirty-minute timer expires or when all participants have declared readiness to conclude. In both cases, the system delivers a voice announcement via TTS and transitions to the Conclusion phase, where participants cast their votes.

Once all votes are collected, the session moves to Closed. The system generates a final report through an LLM call—including session duration, turns per participant, AI intervention rate, and voting results—saves it alongside the moderator’s accumulated summary, and cleans up all associated Redis keys.

7.4.2 Speaking Turn Flow

This is the central data flow of the system, describing what happens from the moment a participant requests to speak until the floor is open for the next speaker.

The flow begins when a participant sends a turn request through the turns WebSocket. **TurnManager** validates that the session is active, no one else is speaking, and moderation is not in progress. If valid, the turn state transitions to Human Speaking, and a notification event is broadcast to all clients. The WebRTC consumer receives this event, updates its local state, sets the speaker in the audio hub, and activates ASR gating—starting the transcription stream for this user only.

While the participant speaks, audio frames arrive through the WebRTC connection. Each frame is resampled to 48 kHz mono 16-bit PCM and processed in two parallel paths: it is forwarded by the audio hub to all other participants' outbound tracks (excluding the speaker), and it is fed to the ASR worker, which sends audio chunks to Azure Speech-to-Text and caches the final transcript segments in Redis.

When the participant ends their turn, the moderation pipeline takes over. The consumer first waits briefly (up to 1.2 seconds) for the ASR cache to stabilize—polling until two consecutive reads return the same result—then collects the final transcript. If the ASR cache is empty, it falls back to the transcript sent by the frontend. The moderation lock is set, blocking new turn requests. The orchestrator evaluates triggers and calls the LLM. Static messages are delivered first (text-only or with TTS). If the LLM decides the AI should speak, a full AI turn is executed: the turn state transitions to AI Speaking, the TTS service synthesizes the message into audio chunks streamed to all participants through the audio hub, and the turn returns to Idle. Finally, if a participant had reserved the floor during the previous turn, an eight-second priority window opens for them.

7.4.3 Audio Flow

All audio flows through the server rather than directly between participants, enabling ASR gating and AI audio injection.

When a participant speaks, their audio arrives via WebRTC and is normalized to a common format: 48 kHz, mono, 16-bit PCM. The server buffers incoming frames and generates fixed 20-millisecond chunks for consistent timing. If the buffer lacks sufficient data, silence is padded; if accumulation exceeds one second, older samples are discarded to reduce latency.

The **SessionAudioHub** forwards each chunk to all other participants' outbound tracks without mixing or processing. The current speaker is automatically excluded from receiving their own audio, preventing echo without requiring client-side cancellation.

When the AI moderator speaks, Azure TTS synthesizes the message in the same 48 kHz format. The server injects these chunks directly into all participants' outbound tracks with slight pacing between chunks to prevent buffer underflow. Since the AI is not a registered peer, no exclusion filter is applied.

8. Value Proposition

8.1 Value for Users

AIutami provides value by addressing a fundamental problem in group discussions: unequal participation. In typical conversations, a few vocal individuals tend to dominate while others remain silent—not because they lack ideas, but because they struggle to find space to speak. This imbalance leads to missed insights, disengaged participants, and decisions that reflect only a subset of the group’s collective knowledge.

AIutami solves this through structured turn-taking managed by an AI moderator. The system monitors who has spoken and for how long, gently prompting quieter members to contribute while preventing any single participant from monopolizing the conversation. Periodic summaries keep everyone aligned on what has been discussed, and participation tracking makes imbalances transparent to the group.

The result is a more democratic conversation where all voices are heard. Users no longer need to compete for attention or worry about being interrupted. The AI handles the social friction of turn-taking, freeing participants to focus on the content of the discussion rather than the mechanics of who speaks next.

8.2 Why It’s a Good Solution

AIutami’s design choices directly address the limitations identified in current research and existing products.

Voice-first approach. While most LLM-based moderation research focuses on text chat, real group discussions happen through voice. AIutami bridges this gap by integrating speech-to-text transcription with AI moderation, enabling intelligent facilitation in the natural medium of spoken conversation.

Proactive intervention. Unlike transcription tools that passively record, AIutami’s moderator actively participates. It can prompt quiet members, redirect off-topic discussions, and summarize progress—intervening when helpful rather than merely observing.

Structured state management. Research shows that LLMs struggle with long, multi-turn conversations, losing track of context and constraints over time. AIutami addresses this through explicit turn tracking and conversation state stored in Redis, ensuring the moderator maintains coherent awareness of who has spoken, what has been discussed, and where the conversation stands.

Task-aware moderation. Studies indicate that generic, task-agnostic moderators produce suboptimal interventions. AIutami allows session-specific context and goals, enabling the AI to provide relevant guidance rather than generic facilitation.

Balanced automation. The system automates the mechanics of turn-taking without replacing human agency. Participants remain in control of the content; the AI only man-

ages the structure. This balance preserves natural discussion dynamics while eliminating the friction of manual coordination.

These choices result in a system that is technically grounded in current research while addressing practical gaps in existing tools.

8.3 Competitive Landscape

Existing solutions address fragments of the group discussion problem, but none combines real-time voice with active AI moderation.

Video conferencing platforms like Zoom and Microsoft Teams provide the communication infrastructure and have recently integrated AI assistants for meeting summaries and Q&A. However, turn-taking remains entirely manual—participants must self-regulate who speaks and when, often resulting in interruptions and unbalanced participation.

Transcription services such as Otter.ai offer real-time speech-to-text and live summaries, but operate as passive observers. They document what is said without influencing how the conversation unfolds.

Conversation intelligence platforms like Gong provide sophisticated analytics and even real-time coaching, but target sales contexts rather than collaborative group discussions.

Alutami occupies a distinct position: it is the only platform designed specifically for structured group discussions with an AI moderator that actively manages turn-taking, monitors participation balance, and intervenes when the conversation needs guidance. Rather than analyzing discussions after they end or documenting them passively, Alutami shapes conversational dynamics as they happen—helping groups reach clear, shared outcomes.

9. Discussion and Future Work

9.1 Critical Reflection

Developing Alutami revealed both the potential and the complexity of real-time AI-moderated voice discussions.

Two technical areas proved particularly demanding. The audio pipeline required significant iteration: WebRTC signaling between browser and server introduced unexpected edge cases, and the audio encoding initially produced distorted, robotic output. The root cause was a mismatch in sample rate and format conversion—the server expected a specific frequency that did not match the browser’s capture settings. Resolving this required careful debugging of the entire audio chain, from browser MediaRecorder through the server-side processing to the forwarding mechanism.

The second challenge was calibrating the AI moderator’s intervention behavior. An overly active moderator disrupts natural conversation flow; an overly passive one fails to provide value. Finding the right balance required iterative refinement of trigger thresholds, cooldown timers, and LLM prompt engineering. The final solution combined rule-based triggers for predictable situations—periodic summaries, silence detection, session time limits—with LLM judgment for nuanced cases like participation imbalance or off-topic drift.

Several architectural decisions proved effective. Separating ephemeral state in Redis from persistent data in PostgreSQL enabled low-latency turn management without sacrificing durability. The service-oriented design kept business logic testable independently from WebSocket consumers. The trigger system’s separation from LLM calls allowed deterministic behavior for time-sensitive situations while preserving AI flexibility for subjective moderation decisions.

These experiences reinforced that real-time voice applications demand tight integration between components often developed separately. The moderation system cannot be designed in isolation from the audio pipeline—intervention timing depends on accurate turn state, which depends on reliable audio processing. Building these systems incrementally with continuous testing proved essential.

9.2 Limitations

The current implementation presents several constraints tied to its architectural choices and development scope.

The system runs on a single server that handles all WebRTC connections, audio forwarding, ASR streams, and LLM calls. This architecture suits small-group discussions but would require horizontal scaling or infrastructure redesign to support multiple concurrent sessions or larger participant counts.

The AI moderator is currently tailored to the Murder Mystery activity. Prompts, trigger conditions, and intervention strategies are designed for this specific collaborative scenario. Applying the system to different discussion formats would require adapting these components.

The platform depends on Azure cloud services for speech recognition, language model inference, and text-to-speech synthesis. While fallback responses handle temporary service unavailability, extended outages would degrade the moderation experience. Response latency is also inherently tied to LLM processing time, which varies based on model load and prompt complexity.

9.3 Future Developments

Two main directions guide the evolution of AIutami.

The first is generalizing the platform beyond the Murder Mystery scenario. The current implementation demonstrates that AI moderation works effectively in structured collaborative tasks, but the same principles apply to study groups, team retrospectives, workshops, and other discussion formats. This generalization requires abstracting the prompt templates and trigger conditions into configurable profiles, allowing facilitators to define session goals, discussion phases, and intervention strategies suited to their context. A session creation interface could offer preset templates while enabling custom configurations.

The second direction involves making the AI moderator more adaptive. Currently, the moderator follows static rules and prompt-based heuristics. Future versions could incorporate learning from session outcomes—adjusting intervention thresholds based on what proved effective, recognizing recurring discussion patterns, and adapting to group dynamics over time. Feedback mechanisms allowing participants to rate AI interventions would provide training signal for such improvements. The goal is a moderator that becomes more effective as it accumulates experience across sessions.

Bibliography

- [1] M. Diehl and W. Stroebe, “Productivity loss in brainstorming groups: Toward the solution of a riddle,” *Journal of Personality and Social Psychology*, vol. 53, no. 3, pp. 497–509, 1987. DOI: 10.1037/0022-3514.53.3.497.
- [2] G. Stasser and W. Titus, “Pooling of unshared information in group decision making: Biased information sampling during discussion,” *Journal of Personality and Social Psychology*, vol. 48, no. 6, pp. 1467–1478, 1985. DOI: 10.1037/0022-3514.48.6.1467.
- [3] G. Stasser and D. D. Stewart, “Discovery of hidden profiles by decision-making groups: Solving a problem versus making a judgment,” *Journal of Personality and Social Psychology*, vol. 63, no. 3, pp. 426–434, 1992. DOI: 10.1037/0022-3514.63.3.426.
- [4] M. Mao, P. Ting, Y. Xiang, M. Xu, J. Chen, and J. Lin, “Multi-user chat assistant (muca): A framework using llms to facilitate group conversations,” Oct. 2024, arXiv:2401.04883v4. arXiv: 2401.04883 [cs.CL].
- [5] F. Dubini, “Multiuser conversational agents based on large language models,” Academic year 2023–2024. Advisor: Prof. Franca Garzotto. Co-advisors: Prof. Enrico Costanza; M.Sc. Mathyas Giudici., Tesi di Laurea Magistrale, Politecnico di Milano, 2024.
- [6] Anonymous Author(s), “Large language models as facilitators in multi-user conversations: A mixed method study,” in *Proceedings of CHI 2026 (manuscript submitted to ACM; anonymized version)*, Anonymized manuscript. Final metadata pending publication., New York, NY, USA: ACM, 2026, pp. 1–23.
- [7] P. Laban, H. Hayashi, Y. Zhou, and J. Neville, “Llms get lost in multi-turn conversation,” May 2025, arXiv:2505.06120v1. arXiv: 2505.06120 [cs.CL].
- [8] H. Sacks, E. A. Schegloff, and G. Jefferson, “A simplest systematics for the organization of turn-taking for conversation,” *Language*, vol. 50, no. 4, pp. 696–735, 1974. DOI: 10.2307/412243.